

[← back to blog](#)

A Lot Can Happen Over Coffee: Hacking Café Coffee Day

July 18, 2026

Café Coffee Day's tagline is "A Lot Can Happen Over Coffee." They weren't kidding. Over two evenings in July 2026, working from nothing but curiosity and a copy of their Android app, I found a chain of vulnerabilities that, stitched together, handed me the phone numbers, emails, invoices and eventually the *accounts* of what looked like every CCD customer in the country. All of it starting from a file anyone can download off the Play Store.

This was independent research, done in my own free time, the same way I've poked at other systems before. Nobody hired me and nobody asked me to look. Fittingly, a good chunk of it happened while I was sitting *inside* a Café Coffee Day, the outlet on the IIT Kanpur campus, nursing a cappuccino and prodding the very app I'd just used to order it. I was just curious how a coffee app that quietly holds millions of people's data actually works under the hood, and the answer turned out to be "not very carefully."

What makes this one different from some of my earlier writeups, like the critical vulnerabilities I found in CBSE's On-Screen Marking portal, is that it has a genuinely happy ending. Where that disclosure turned into months of silence and public denial, this one didn't. I reported everything to CERT-In, they forwarded it to the company, and the whole chain was **fully patched within ten days**. I then went back and re-verified every fix myself before writing a word of this. In this line of work that kind of turnaround is rare enough to be worth calling out, so: credit where it's due.

downloaded in bulk, and in every case I stopped the moment a flaw was proven rather than pushing further. All of the account-takeover and OTP testing was done against my own number and my own account. Noticing that a door has been left wide open is not the same thing as walking through it.

Timeline

DATE	EVENT
07 Jul 2026	Pulled apart the CCD Android app (<code>com.ccd.app.global</code> v1.0.12) out of pure curiosity.
08 Jul 2026	Realised how bad it was and reported the full chain to CERT-In. Registered under Ref: CERTIn-51351626 .
18 Jul 2026	CERT-In relayed that the company had confirmed every vulnerability was fixed.
18 Jul 2026	Independently re-tested each finding against production — all confirmed remediated . Published this writeup.

What is CCD?

Café Coffee Day is one of India's largest coffee-shop chains, with hundreds of outlets across the country. Like most retail brands it has a mobile app: you use it to order ahead, pay, collect loyalty points, and store your profile. The app is a native Kotlin (Jetpack Compose) application, and its backend lives on `*.reciproci.com`, a platform built by the vendor TechTree IT.

Anything that stores millions of customers' phone numbers, order histories and payment details is worth looking at carefully. So I pulled the APK apart.

The app ships its logic in compiled Kotlin/Java, so step one was to rip the APK open with JADX and start reading. This is the unglamorous part nobody puts in the movies: thousands of files of obfuscated, single-letter-named classes, and you just... read, until the shape of the thing appears. Two things jumped out almost immediately.

The first was that the app "encrypts" every request and response body. Every API call goes out wrapped like this:

```
{ "requestData": "aGVsbG8gdGhlcmUgZnJpZW5k ... " }
```

The second was *how* it encrypts them. The entire scheme hangs on a single AES key, and that key is baked directly into the app.

Static analysis only gets you so far, though. To actually watch the app talk to its backend I set up a full dynamic rig: a rooted Android emulator, [Frida](#) for runtime instrumentation, and a set of hooks to defeat the app's certificate pinning and dump traffic in the clear. Frida lets you reach into a running process and rewrite it live, so I could hook the app's own crypto routines and print every request *before* it got encrypted and every response *after* it got decrypted. From that point on, the app's "encryption" wasn't slowing anything down; I was reading its mail in plaintext as it was written. (This rig came back to save me later, too, when the fixed build tried to make my life harder.)

The Client-Side Crypto Was Theatre

Buried in a utility class (`AppUtil.k()`) was the app's master key, lightly obfuscated by an array that just swaps adjacent byte pairs, which is trivially reversible. Undo the swap and you get a **hardcoded AES-128-CBC key that is identical across every install**. To make matters worse, the same value is reused as the IV.

```

# Recovered from AppUtil.k() – the "obfuscation" just swaps
# adjacent byte pairs, so it's trivially reversible.
# ONE hardcoded AES-128-CBC key, identical across every install.
from Crypto.Cipher import AES
import base64

KEY = b"wYzBxWfu8L2RBBxg" # ships inside every APK

def decrypt(b64):
    c = AES.new(KEY, AES.MODE_CBC, KEY) # IV == KEY
    pt = c.decrypt(base64.b64decode(b64))
    return pt[:-pt[-1]] # strip PKCS#7 padding

# Every "encrypted" body is now readable – and forgeable – offline.

```

That single key protects *all* API traffic, the local token store, and even the phone-number "encryption" used during login. Because it ships inside the app, anyone can extract it, decrypt captured traffic, and forge or tamper with any encrypted payload offline. A secret that every user already has on their phone isn't a secret.

On its own, that's a "defense-in-depth is broken" finding. The real problem was that this encryption was the *only* thing standing in front of the API, and several endpoints didn't even require it. Many accepted plain, unencrypted JSON bodies directly. The wrapper wasn't a security control; it was a speed bump that kept casual observers from noticing what the API actually did.

Finding #1: A Wide-Open S3 Bucket

I hadn't even started on the API properly when the storage layer handed me the single biggest finding of the night, no exploitation required.

The app pulls its images out of an Amazon S3 bucket, `ccd-multimedia.s3.ap-south-1.amazonaws.com`. Standard stuff, so almost as a reflex I pointed a plain `curl` at it to see if it would list. It shouldn't have. It did. The bucket was configured with a public `AllUsers: READ`

login screen, no token, no clever trick. Just a URL.

```

$ curl -s 'https://ccd-multimedia.s3.ap-south-1.amazonaws.com/\
?list-type=2&prefix=wsstore/segmentmember/'

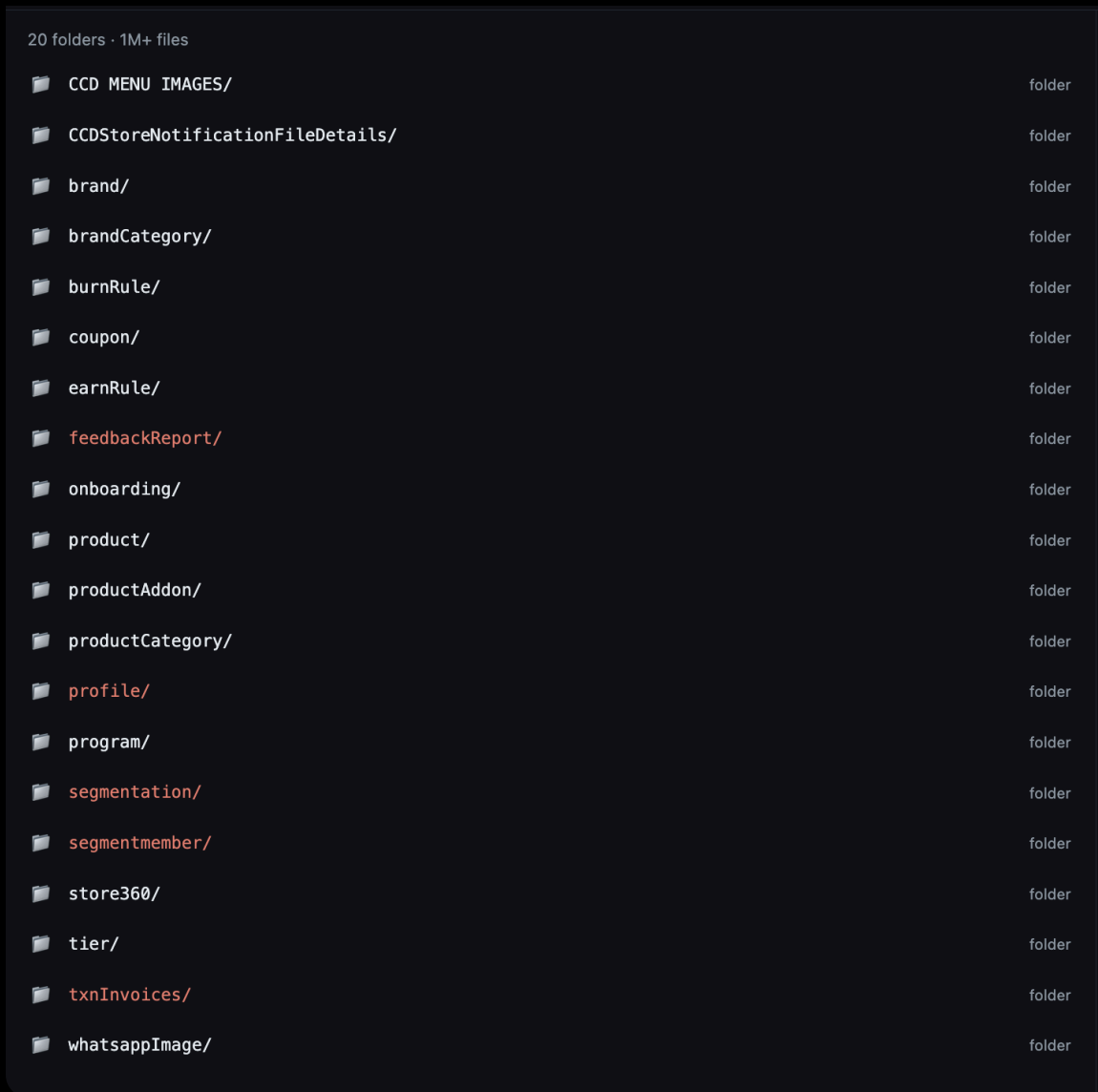
<Key>wsstore/segmentmember/MemberId_with_mobileNo_20241127.csv</Key>
<Key>wsstore/segmentmember/MemberId_with_mobileNo_20241203.csv</Key>
... 91 CSV files · ~62 MB · column "Mobile_Number"
    full of live 91xxxxxxxxxx customer numbers

# No credentials. No signature. Bucket ACL grants AllUsers: READ.
```

And it was not storing coffee-cup photos. Paginating through the listing turned up exactly the kind of thing that makes your stomach drop a little:

- **wsstore/segmentmember/** — 91 CSV exports (~62 MB) with names like **MemberId_with_mobileNo_*.csv**. These were chunks of the marketing database, each containing a **Mobile_Number** column full of live **91xxxxxxxxxx** customer numbers.
- **wsstore/profile/** — over 15,000 customer profile photos (~4.7 GB, and still truncating).
- **wsstore/txnInvoices/** — more than 3,000 customer tax-invoice PDFs (name, itemised order, amounts, GST).
- **wsstore/feedbackReport/** and **wsstore/segmentation/** — feedback and customer-segmentation exports.

To make sense of what was in there without blindly downloading anything, I threw together a small read-only browser that just lists the bucket and previews objects in place. It reported over a million files across twenty folders, which is roughly the moment "misconfiguration" turns into "the whole customer estate":



The bucket's ACL was itself world-readable, so I could literally read the misconfiguration that caused all this back off the server: **AllUsers: READ** , in black and white. To its small credit, the exposure was read-only. An anonymous **PUT** and **DELETE** both came back **403** , so nobody could quietly swap someone's invoice for a malicious PDF or wipe the bucket. All you could do was download every last byte of it. Which, when the "it" is the phone numbers, photos and invoices of what appears to be the entire customer base, is more than enough. This was a full customer-data breach that didn't even need the app installed, and I hadn't yet touched a single API endpoint.

Finding #2: Downloading Anyone's Invoice, No Login Required



```
# NO Authorization header. Just a plaintext txn0id.  
$ curl -s -X POST https://ccd-api.reciproci.com/api/unsec/v1/invoice/download \  
  -H 'Content-Type: application/json' \  
  --data '{"txn0id":"5370342"}'  
  
{ "invoicePath": "https://ccd-multimedia.s3.../wsstore/txnInvoices/<uuid>.pdf" }  
  
# txn0id is a sequential integer: ~1,000,000 -> 5,370,000+  
# i.e. millions of invoices, every one enumerable, no login required.
```

With no **Authorization header at all**, you send a plaintext `{"txn0id": <id>}` and get back a public S3 URL to that customer's tax-invoice PDF. The `txn0id` is a simple **sequential integer**; the values I saw spanned from around 1,000,000 up past 5,370,000, i.e. millions of orders, each one a valid, enumerable invoice. A sibling endpoint, `invoice/view`, returned the same data as structured JSON, which is even easier to scrape.

Opening one of those `txnInvoices` PDFs straight from the bucket gives you a complete tax invoice — store, date, itemised order, taxes, amounts, and the customer's name (redacted here; the numbers were left intact only to show the flaw):

ccd-multimedia / wsstore / txninvoices

File Name	Size	Date
035cd14f-ccd4-4364-bec4-e4d8df48f4e7.pdf	7.1 KB	2025-10-31
03b53c93-1c0c-4340-898a-886050e9e340.pdf	7.2 KB	2025-10-03
03c603c3-7377-44ee-b59a-b1b91c25d77.pdf	7.0 KB	2025-11-22
03d3aeb4-b6e3-49e0-ad96-bc86bfae72f6.pdf	7.2 KB	2026-01-26
03d4c019-85d4-4f5f-b64e-74e4e3940890.pdf	7.1 KB	2025-07-09
03f39784-5c17-45c9-ae4b-e29159fcc9ff.pdf	7.1 KB	2026-11-25
040ddb9d-d9c3-47b0-a71a-e7ae457d9b24.pdf	6.9 KB	2025-08-26
0417ae9e-307a-4b29-8ffe-de0a6231571a.pdf	7.0 KB	2026-07-04
0486a0a8-94eb-4a54-a011-aa6acd3898ef.pdf	7.1 KB	2026-06-01
04957a96-2939-4d79-a0ed-6d0abfd7bee2.pdf	7.0 KB	2025-07-02
04c55078-67f1-44de-0953-c44e59cfa810.pdf	7.1 KB	2026-12-21
05160cc3-5cad-48b2-b263-805f8f8cead0.pdf	6.9 KB	2025-07-31
0519a1a5-72d3-4707-9141-290cddb22973.pdf	7.1 KB	2025-11-08
051d0973-c5d1-4a19-abd1-a0fffb3ad22b.pdf	7.2 KB	2026-05-01
0522abd7-b04a-4eeb-b259-331ea204c952.pdf	7.0 KB	2026-10-14
05263e15-6035-4213-9f5c-c03150093e9c.pdf	7.1 KB	2025-09-01
05506710-2d16-4d32-92f9-5cfec05563bb.pdf	7.2 KB	2025-12-16
05648bd9-9649-42d4-82a4-aa07781df127.pdf	7.2 KB	2026-03-10
057f56b6-8a84-49a5-8632-2abe2fb7a617.pdf	7.0 KB	2026-10-12

040ddb9d-d9c3-47b0-a71a-e7ae457d9b24.pdf
wsstore/txninvoices/040ddb9d-d9c3-47b0-a71a-e7ae457d9b24.pdf · 6.9 KB · 2025-08-26T08:43:33.000Z

Download original

Document 1 / 1 57% +

1

COFFEE DAY GLOBAL LIMITED
PONDIA MARKET CITY CHENNAI
Pondia, Pondicherry, TN, 605 006 (India) Phone: +91 4382 408042
PIN NO: 605001 (INDIA)
PONDIA, INDIA

TAX INVOICE

BILL NO: [REDACTED] DATE: [REDACTED] TIME: 08:43 PM
CUSTOMER: [REDACTED]

SL. NO.	DESCRIPTION	QTY	AMOUNT
1	CLASSIC NEW YORK CHOCOLATE CAKE	1	11.00
TOTAL			11.00
CASH			11.00
NET			11.00
NET AMOUNT			11.00
NET AMOUNT RECEIVED			11.00

PAY MODE: [REDACTED]
PROVIDER STATIC QR: [REDACTED]
Bank: [REDACTED]

THANKS AND VISIT AGAIN

So already, before ever creating an account, an attacker could walk that sequential `txn0id` and harvest every CCD customer's name, order details and amounts. That alone is a serious breach, but an invoice doesn't carry a phone number, and a phone number was what I needed to turn this into something worse. The next endpoint gave me exactly that.

Finding #3: The Endpoint That Handed Out Phone Numbers

This is the one that took an ugly data-exposure story and turned it into a full account-takeover story.

Deeper in the API sat `POST /api/ns/order/v6/get`, which returns the full details of an order. This one actually asks for a Bearer token, so at first glance it looks like the grown-up in the room. The catch is that "a token" is *all* it wants. It checks that you're logged in as *somebody*; it never checks that the order you're asking about is *yours*.

That's a textbook **Broken Object Level Authorization** flaw (a BOLA, cousin of the classic IDOR), and there's a simple way to test for it: log in as yourself, then ask for something that isn't yours and see if the server says no. So I made a throwaway account, looked up my own order id, and then just... subtracted one.

```

# Signed in as MY throwaway account (customerId 2138329).
# Just increment the txn0id to an order that isn't mine...
$ curl -s -X POST https://ccd-api.reciproci.com/api/ns/order/v6/get \
  -H 'Authorization: Bearer <my token>' \
  --data '{"txn0id":5370341}'

{
  "customerDetails": {
    "customerId": 2192217,          <-- not me
    "name":      "J*****n",
    "phone":     "9163XXXXX74",
    "email":     "j****@****.com",
    "tierName":  "RED"
  }
}
# One token. Any order id. No ownership check. -> BOLA / IDOR.

```

My own id gave me my own order, as it should. The very next id down gave me a *stranger's* order, and sitting inside it was a `customerDetails` block with their **name, phone number, email address and loyalty tier** printed in plaintext, and for delivery orders, the recipient's full street address and mobile number too. Every id I tried resolved to a real person who was not me, every one of them read with my one throwaway token. It's a genuinely strange feeling to type a number, hit enter, and have a stranger's phone number scroll up your terminal.

Now recall that `txn0id` is the *exact same* sequential number that Finding #2 hands out to anyone, no login required. That's the moment the individual bugs stop being bugs and become a machine: enumerate the order space unauthenticated, pipe each id into `order/v6/get` behind one disposable account, and you can walk the entire customer base pulling names, phones and emails on a loop. I stopped at a tiny handful of ids, just enough to be certain it was real. The point of a proof of concept is to prove it, then put the gun down.

Finding #4: Account Takeover via Unthrottled OTP Brute-Force

phone number. This finding is what connected it to someone's actual account.

CCD logs you in with an OTP, driven by two unauthenticated endpoints: `customer/v4/verification` texts a code to a number and hands back a `referenceId`, and `otp/v5/verify` checks a code against that reference. Two facts turned that into a brute-force target. First, the OTP is only **5 digits** — a hundred thousand possible codes, which sounds like a lot until you remember computers count fast. Second, and this is the fatal one, `otp/v5/verify` had **no rate limiting, no lockout, and no CAPTCHA of any kind**.

There's really only one way to confirm something like that, so I pointed it at my own number and my own reference and fired 60 deliberately wrong codes at it, back to back, to see when it would flinch:

```

# Brute-forcing the OTP against my OWN referenceId.
# 5 digits = 100,000 values. Tested against my own number only.
$ for otp in $(seq -w 0 99999); do
    curl -s .../api/unsec/customer/otp/v5/verify \
        --data "{\"otp\":\"$otp\",\"referenceId\":\"$ref\"}"
done

400 "Sorry its an invalid OTP"      # attempt 1
400 "Sorry its an invalid OTP"      # attempt 2
...                                # 60 wrong guesses, back to back
400 "Sorry its an invalid OTP"      # attempt 60

# No lockout. No throttle. No CAPTCHA. referenceId still valid.
```

It never flinched. All sixty came back `400 "Sorry its an invalid OTP"`, and the `referenceId` was still happily valid at the end of it. No slowdown, no lockout, no "too many attempts, try later." A correct guess, meanwhile, returns a complete session: `accessToken`, `refreshToken`, `customer_id`. So the whole login reduces to a race

take the account, without the victim's phone ever buzzing in a way they'd notice.

Stitching It Together

Individually these are serious. Chained, they're an unauthenticated, victim-agnostic mass account-takeover primitive:

- **Finding #2** gives you unauthenticated, sequential enumeration of every order ID.
- **Finding #3** turns each order ID into a real customer's phone number, with one throwaway token.
- **Finding #4** turns any phone number into an unthrottled path *toward* that account.

To be precise about that last step: Finding #4 doesn't magically hand you the account. It's a brute-force, so you still have to grind through the OTP space to actually get in. But that's exactly the point of a rate limit, and there wasn't one. A 5-digit code with no lockout and no throttle is a finite, parallelisable guessing game that the server will patiently let you keep playing until you win. It's not a guaranteed instant takeover; it's an open door with a numeric lock and unlimited attempts, which in practice is almost as bad.

So chain the three together and the shape is unmistakable: start from nothing, pick an ID range, and what comes out the other end is a working route into stranger after stranger's account, with no prior knowledge of any target required. That's the whole thing, and it's why I stopped poking and wrote it up to CERT-In the same day I finished mapping it.

What Was Actually Solid

driving a real ₹90 order and replaying it with a tampered `billAmount` of ₹1; the server ignored the client amount, recomputed the price from the catalogue, and generated its own payment hash. Cart pricing is server-authoritative, so "pay ₹1 for a ₹90 order" doesn't work. The JWT was also robust (`alg:none` , junk signatures, and tampered payloads were all rejected), the mobile-number change was blocked server-side, and the delivery-address endpoints correctly enforced ownership. The problems were concentrated in the read paths and the OTP flow, not everywhere.

Responsible Disclosure

I reported the full chain to **CERT-In** on 8 July 2026, and it was registered under **Ref: CERTIn-51351626**:

Dear Sir/Madam,

Thank you for reporting this incident to CERT-In. We have registered your complaint/incident under Ref: CERTIn-51351626.

They asked for a detailed, timestamped, step-by-step PoC for each finding, which I put together as a single runbook that prints both a local clock and the server's own `Date:` header at every step, so the evidence is self-authenticating. That went back to them along with screen recordings.

Then, on 18 July, the reply I wish every disclosure ended with:

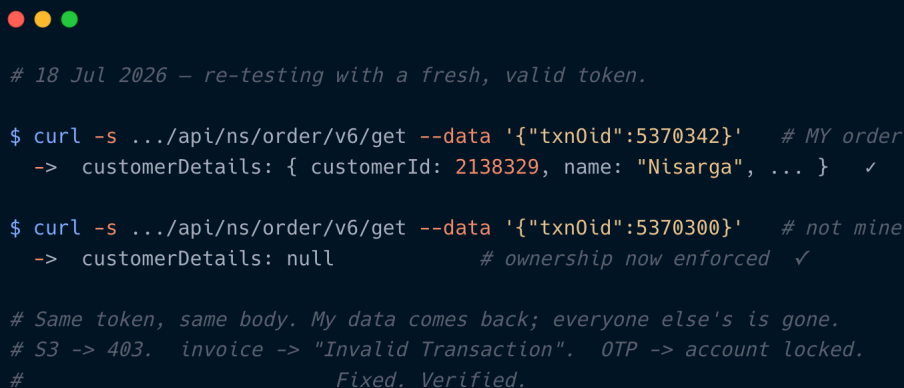
With reference to trailing mail, the concerned organization has confirmed that the reported vulnerabilities are fixed. You are requested to verify at your end and confirm.

Verifying the Fix

I don't take "it's fixed" on faith, so I re-tested everything against production the same day. The unauthenticated findings were easy to

hardened build (v1.0.13) that made grabbing one genuinely annoying.

This is where my old dynamic rig earned its keep. The new build had stopped stashing the token in plain reach and moved it into Android's Keystore-backed encrypted storage, and it had re-obfuscated its networking libraries so the class names I used to hook were gone. So I went back to Frida, attached to the freshly installed app, logged in with my own number, and hooked the one spot that can never really be hidden: the function that builds the outgoing HTTP request. The moment the app made its first authenticated call, my hook printed the `Authorization: Bearer ...` header straight to my terminal. Token in hand, I fired the exact same BOLA request that used to leak strangers' phone numbers.



```

# 18 Jul 2026 - re-testing with a fresh, valid token.

$ curl -s .../api/ns/order/v6/get --data '{"txn0id":5370342}' # MY order
-> customerDetails: { customerId: 2138329, name: "Nisarga", ... } ✓

$ curl -s .../api/ns/order/v6/get --data '{"txn0id":5370300}' # not mine
-> customerDetails: null # ownership now enforced ✓

# Same token, same body. My data comes back; everyone else's is gone.
# S3 -> 403. invoice -> "Invalid Transaction". OTP -> account locked.
# Fixed. Verified.

```

- **The S3 bucket is locked down.** Anonymous listing, the ACL read, and even a direct GET on a previously-public object key all now return `403 AccessDenied`. Block Public Access is clearly in effect.
- **The unauthenticated invoice endpoints reject arbitrary IDs.** Every `txn0id` I tried across the whole range now returns `400 "Invalid Transaction"`.

an empty skeleton with `customerDetails: null`. Ownership is enforced server-side. The record that used to hand me a stranger's phone and email now hands me nothing.

- **The OTP flow is throttled.** Where 60 wrong guesses previously sailed through, the endpoint now enforces an account-level lockout ("Too many OTP tries... paused for today").

There was even some unprompted hardening I hadn't asked for. The newer app build (v1.0.13) had migrated its at-rest token storage from the old static-key scheme to Android's Keystore-backed `EncryptedSharedPreferences`, tightened its obfuscation, and added a set of security headers (`HSTS`, `X-Frame-Options`, `X-Content-Type-Options`) to the API. That's a team that actually took the report seriously.

Takeaways

Every finding here traces back to the same handful of fundamentals:

- **The client cannot be trusted, and it certainly can't hold secrets.** A key shipped in every APK is not a key. Encryption performed on the attacker's own machine is not a control.
- **Object existence is not authorization.** Every endpoint that returns or modifies data must check *who is asking*, derived from the authenticated session, not just *what* they asked for. `order/v6/get` had a token check but no ownership check, and that gap was the whole ballgame.
- **Sequential IDs plus missing authorization is a data breach waiting to happen.** If your record identifiers are guessable, every read endpoint over them needs to be airtight.
- **OTP endpoints need rate limits.** A 5-digit code with no throttle is a 100,000-guess door with no lock.

database.

None of these are advanced defenses; they're the fundamentals, the kind of thing you'd expect any team handling millions of people's data to have in place from day one. What ultimately made this story end well wasn't clever engineering on the fixes, it was that someone on the other side actually read the report and acted on it quickly. After a few disclosures where that didn't happen, it's genuinely nice to be able to write one where it did.

Thanks for reading.